



EAST WEST UNIVERSITY

CSE207: Data Structures

Section:10

Semester: Summer 2025

Project Report

Hospital Patient Management

Submitted by:

Student ID	Student Name
2023-3-60-148	Kamrul Hasan
2023-1-60-190	Mahanur Nesa Tanni
2023-2-60-293	Md. Jubaer Ahmmed
2024-1-60-094	Moshiur Rahaman

Submitted To:

Dr.Md. Manowarl Islam,
Associate Professor, Department of CSE,
East West University

Submission Date: 26/08/2025

Table of Contents

1	Introduction
2	Objective
3	Problem Statement
4	Data Structures Used and their justification
5	Methodology and Implementation
6	Test Cases and Results
7	Challenges and Limitations
8	Conclusion
9	Appendix

Introduction:

The project, "Hospital Patient Management System," is developed by Data Structure. It is designed to show how a hospital handles patient lines using linked lists and queues. The system handles both regular and emergency patients, managing their details, updates, and discharge status.

Objective:

1. To implement data structures (linked lists, queues) in a practical way.
2. To handle hospital patients based on their priority number (emergency and normal).
3. To keep patient info updated, remove it when needed, and track it effectively.
4. To create a realistic simulation of hospital queue management.

Problem Statement:

In a hospital, managing patients efficiently is critical. Patients may have different priorities, such as emergencies or regular checkups. A manual process may be error-prone and inefficient. This project aims to develop a system that:

1. Registers patients with different priorities.
2. Keeps separate queues for normal and emergency cases.
3. Allows patient search, update, deletion, and discharge.
4. Displays treated and untreated patients separately.

Data Structures Used and Their Justification:

Linked List:

1. Used to store the main list of all patients (PatientList) and discharged patients.
2. Efficient insertion and deletion of patients from the middle of the list.

Justification: Dynamic memory management and frequent insert/delete operations.

Queue (Singly Linked):

- 1.Used for managing the normal appointment queue (AppointmentQueue).
- 2.First-Come, First-Serve (FCFS) model or FIFO (First-in First-Out).

Priority Queue (Sorted Linked List):

- 1.Used for emergency patients (EmergencyQueue).
- 2.Patients with lower priority numbers (more critical) are served first.

Justification: Ensures critical patients are treated with urgency.

Node Structure:

- 1.Wraps around the Patient object to allow flexibility and queue linkage.

Methodology and Implementation:

The Hospital Patient Management System was built using object-oriented programming in C++ and data structures to simulate how a hospital works. Here's an overview of the main parts of the system and what they do, along with some code examples.

Patient Class:

The Patient class holds all the necessary information about a patient such as ID, name, age, disease, doctor, and priority. The class also includes static auto-incrementing IDs to uniquely identify each patient.

PatientList Class:

The PatientList class uses a linked list to manage current and discharged patients. It supports adding, removing, updating, and displaying patients. The list is also used to search for a patient by ID.

AppointmentQueue Class:

This class handles the normal patient queue (non-emergency) using a simple FIFO linked queue. Patients are added to the rear and served from the front.

EmergencyQueue Class:

EmergencyQueue uses a sorted linked list to maintain patients based on their priority. Lower priority numbers indicate more critical conditions and are placed earlier on the list.

Main Function:

The main () function acts as the interface where users can interact with the system. It includes a menu-based loop to handle user input for adding, deleting, updating, and serving patients in both queues.

Functions:

- 1.Add, update, delete patients.
- 2.Search by ID.
- 3.Display all current and discharged patients.
- 4.Serve patients (normal and emergency queues separately).

User Interface:

- 1.Text-based menu in the main () function.
- 2.Prompts users to enter commands and information.

Test Cases and Results:

Add a normal patient:

Add an emergency patient:

Update patient information:

Delete normal/emergency patient:

Serve next patient in both queues:

Show discharged patients:

Search for a patient by ID:

Show patients to be treated:

Show Normal Queue:

Show Emergency Queue:

```
E:\5\CSE207\C++\C++\Project\Project.exe

Hospital Patient Management System
1. Add Patient
2. Delete Patient
3. Search Patient
4. Update patients information.
5. Show patients to be treated.
6. Show Normal Queue
7. Show Emergency Queue
8. Serve Next Emergency.
9. Serve next normal patient
10. Show patients that have been discharged after taking treatment
11. Exit
Choose option:
```

Challenges and Limitations:

- 1. Input Validation:** The system needs to make sure that all the inputs are valid for patient status updates.
- 2. Data Duplication:** There's a risk of duplicating entries if the system relies on manual input.
- 3. User Interface:** The command-line interface might not be easy for all users.
- 4. Data:** The data isn't saved. It's lost when the program closes.

Conclusion:

This project demonstrates how data structures can solve real-world problems like hospital queue management. By managing different patient types and tracking their statuses, it improves efficiency in a simulated healthcare setting. The system successfully applies data structure concepts in practical programming, meeting its core objectives.

Appendix:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Patient {
```

```
public:
```

```
    int id;
```

```
    string name;
```

```
    int age;
```

```
    string disease;
```

```
    string doctorName;
```

```
    int priority; // 0 = normal, >0 = emergency
```

```
    Patient* next;
```

```
    static int nextID;
```

```
    Patient(string n, int a, string d, string doc, int p) {
```

```
        id=nextID++;
```



```

    name=n;
    age=a;
    disease=d;
    doctorName=doc;
    priority = p;
    next = nullptr;
}

Patient(string n, int a, string d, string doc) {
    id=nextID++;
    name = n;
    age = a;
    disease = d;
    doctorName = doc;
    priority=0;
    next = nullptr;
}

Patient(){
    id=0;
    name="";
    age=0;
    disease="";
    doctorName="";
    priority=0;
    next=nullptr;
}

```

```
};
```

```
int Patient::nextID=1;
```

```
class PatientList {
```

```
private:
```

```
    Patient* head;
```

```
    Patient* head2;
```

```
public:
```

```
    PatientList(){
```

```
        head=nullptr;
```

```
        head2=nullptr;
```

```
    }
```

```
    bool isEmpty(){
```

```
        if(head==nullptr){
```

```
            cout<<"No patients in the list.\n";
```

```
            return true;
```

```
        }
```

```
        else{
```

```
            return false;
```

```
        }
```

```
    }
```

```
    void addPatient(Patient* newPatient){
```

```
        newPatient->next = head;
```

```
        head = newPatient;
```

```
}
```

```
void removepatient(int id){  
    if(isEmpty()){  
        return;  
    }  
    Patient* temp=head;  
    Patient* prev = nullptr;  
    while (temp != nullptr && temp->id != id) {  
        prev = temp;  
        temp = temp->next;  
    }  
    if(temp==nullptr) {  
        cout << "Patient not found!\n";  
        return;  
    }  
    if(prev==nullptr)  
        head=temp->next;  
    else{  
        prev->next=temp->next;  
    }  
    delete temp;  
    cout << "Patient deleted.\n";  
}
```

```
void discharged(int id){
```

```

Patient* temp=head;
Patient* prev = nullptr;
Patient* temp2=nullptr;
while (temp!=nullptr&&temp->id!=id) {
    prev=temp;
    temp=temp->next;
}
if (temp==nullptr){
    cout<<"Patient not found!\n";
    return;
}
if (prev == nullptr)
    head = temp->next;
else{
    prev->next=temp->next;
}
temp2=temp;

temp2->next=nullptr;
if(head2==nullptr){
    head2=temp2;
}
else{
    Patient* temp3=head2;
    while(temp3->next!=nullptr){
        temp3=temp3->next;
    }
}

```

```

    }
    temp3->next=temp2;
}
removepatient(id);
}
void showdischarged() {
Patient* temp=head2;
if(isEmpty()){
    return;
}
cout << "\nPatients that have been treated and discharged:\n";
while (temp!= nullptr){
    cout <<"ID: "<<temp->id<< ", Name: "<< temp->name<< ", Age: " << temp->age
        << ", Disease: " << temp->disease<< ", Doctor: " << temp->doctorName
        << ", Priority: "<<temp->priority<< "\n";
    temp = temp->next;
}
}

```

```

Patient* searchPatient(int id) {

```

```

    Patient* temp = head;

```

```

    while (temp != nullptr) {

```

```

        if (temp->id == id)

```

```

            return temp;

```

```

        temp = temp->next;
    }
}

```

```

    }
    return nullptr;
}

```

```

void updatePatient(int id, string newDisease, string newDoctor){

```

```

    Patient* p = searchPatient(id);
    if(isEmpty()){
        return;
    }
    if (p != nullptr) {
        p->disease = newDisease;
        p->doctorName = newDoctor;
        cout << "Patient info updated.\n";
    } else {
        cout << "Patient not found!\n";
    }
}

```

```

void displayAllPatients() {

```

```

    Patient* temp=head;
    if(isEmpty()){
        return;
    }
    while(temp!=nullptr) {
        cout <<"ID: " <<temp->id<< " , Name: " <<temp->name<<" , Age: " <<temp->age
        <<" , Disease: " <<temp->disease<<" , Doctor: " <<temp->doctorName

```

```

        <<" , Priority: "<<temp->priority<<"\n";
        temp=temp->next;
    }
}
};

```

```

class Node {
public:
    Patient* patient;
    Node* next;
    Node(Patient* p){
        patient = p;
        next = nullptr;
    }
};

```

```

class AppointmentQueue {
private:
    Node* front;
    Node* rear;

public:
    AppointmentQueue(){
        front=rear=nullptr;
    }
}

```

```
bool isEmpty(){  
    if(front==nullptr&&rear==nullptr){  
        cout<<"No patients in the normal queue.\n";  
        return true;  
    }  
    else{  
        return false;  
    }  
}
```

```
void addToQueue(Patient* patient){  
    Node* newNode=new Node(patient);  
    if (rear==nullptr){  
        front=rear=newNode;  
    }  
    else{  
        rear->next=newNode;  
        rear=newNode;  
    }  
}
```

```
void serveNextPatient(PatientList &patient){  
    if(isEmpty()){  
        return;  
    }  
    Node* temp = front;
```



```

Patient* p = temp->patient;

front = front->next;

if (front == nullptr)
    rear = nullptr;

cout<<"Has "<<p->name<<" "<<p->id<<" patient been treated here?(Press y for Yes or n for
No):";

char status;
cin>>status;
while(status!='y'&&status!='n'){
    cout<<"Invalid input. Please enter 'y' for Yes or 'n' for No: ";
    cin>>status;
}
if(status=='y'){
    patient.discharged(p->id);//move patient to discharged list
}
if(temp->next!=nullptr){
    p=temp->next->patient;
    cout<<"Next patient to be served: "<<p->name<<" ID: "<<p->id<<endl;
}
else{
    cout<<"No more patient left to be served"<<endl;
}
delete temp;
}

```

```

void displayQueue() {
    Node* temp=front;
    if(isEmpty()){
        return;
    }
    while (temp!=nullptr) {
        cout<<"ID: "<<temp->patient->id<<"", Name: "<<temp->patient->name<<"\n";
        temp=temp->next;
    }
}

void deletePatient(int id,PatientList &patient) {
    if(isEmpty()){
        return;
    }

    if (front->patient->id==id) {
        Node* temp = front;
        front = front->next;
        if (front == nullptr) {
            rear=nullptr;
        }
        delete temp;
        cout << "Patient deleted from queue.\n";
        return;
    }
}

```

```

Node* temp=front;

Node* prev=nullptr;

while (temp!=nullptr&&temp->patient->id != id) {

    prev=temp;

    temp=temp->next;

}

if (temp==nullptr){

    cout<<"Patient not found!\n";

    return;

}

else{

    Patient* p = temp->patient;

    char status;

    cout<<"Has "<<p->name<<" "<<p->id<<" patient been treated here?(Press y for Yes or
n for No):";

    cin>>status;

    while(status!='y'&&status!='n'){

        cout<<"Invalid input. Please enter 'y' for Yes or 'n' for No: ";

        cin>>status;

    }

    if(status=='y'){

        patient.discharged(p->id);

    }

    prev->next=temp->next;

}

```

```

        delete temp;

        cout<<"Patient deleted.\n";
    }

    void updatePatient(int id, string newDisease, string newDoctor){
        if(isEmpty()){
            return;
        }

        Node* temp=front;
        PatientList patient;
        while(temp!=nullptr&&temp->patient->id!=id){
            temp=temp->next;
        }

        if(temp==nullptr){
            cout<<"\npatient not found"<<endl;
            return;
        }

        temp->patient->disease=newDisease;
        temp->patient->doctorName=newDoctor;
        patient.updatePatient(id,newDisease,newDoctor);
    }
};

```

```

class EmergencyQueue{
private:
    Node* head;

```

public:

```
EmergencyQueue(){
```

```
    head = nullptr;
```

```
}
```

```
bool isEmpty(){
```

```
    if(head==nullptr){
```

```
        cout<<"No patients in emergency queue.\n";
```

```
        return true;
```

```
    }
```

```
    else{
```

```
        return false;
```

```
    }
```

```
}
```

```
void addEmergencyPatient(Patient* patient) {
```

```
    Node* newNode = new Node(patient);
```

```
    if (head == nullptr || patient->priority<head->patient->priority) {
```

```
        newNode->next=head;
```

```
        head=newNode;
```

```
    } else {
```

```
        Node* temp = head;
```

```
        while (temp->next != nullptr && temp->next->patient->priority<=patient->priority) {
```

```
            temp = temp->next;
```

```
        }
```

```
        newNode->next = temp->next;
```

```

        temp->next = newNode;
    }
}

void serveNextEmergency(PatientList &patient) {
    if (isEmpty()){
        return;
    }

    Node* temp = head;
    head = head->next;

    Patient* p = temp->patient;

    cout<<"Has "<<p->name<<" "<<p->id<<" patient been treated here?(Press y for Yes or n for
No):";

    char status;
    cin>>status;

    while(status!='y'&&status!='n'){
        cout<<"Invalid input. Please enter 'y' for Yes or 'n' for No: ";
        cin>>status;
    }

    if(status=='y'){
        patient.discharged(p->id);
    }

    if(temp->next!=nullptr){
        p=temp->next->patient;

        cout<<"Next patient to be served: "<<p->name<<" ID: "<<p->id<<" Priority:
"<<p->priority<<endl;
    }
}

```

```

else{
    cout<<"No more patient left to be served"<<endl;
}
delete temp;
}

void deleteemrgencypatient(string name,int id,PatientList &patient){
    if (isEmpty()){
        return;
    }
    Node* temp=head;
    Node* prev=nullptr;
    while(temp!=nullptr&&temp->patient->name!=name && temp->patient->id!=id){
        prev=temp;
        temp=temp->next;
    }
    if (temp==nullptr){
        cout<<"No such patient found in system."<<endl;
        return;
    }
    cout<<"Patient found!"<<endl;
    char status;

    cout<<"Has "<<temp->patient->name<<" "<<temp->patient->id<<" patient been treated
here?(Press y for Yes or n for No):";

    cin>>status;

    if(status=='y'){
        patient.discharged(temp->patient->id);
    }
}

```

```

    }
    if(prev==nullptr){
        head=temp->next;
    }else{
        prev->next=temp->next;
    }
    delete temp;
}

```

```

void displayEmergencyQueue() {
    Node* temp = head;
    if (isEmpty()){
        return;
    }
    while (temp != nullptr) {
        cout << "ID: " << temp->patient->id << ", Name: " << temp->patient->name << ", Priority: " << temp->patient->priority << "\n";
        temp = temp->next;
    }
}

```

```

void updatePatient(int id, string newDisease, string newDoctor){
    Node* temp=head;
    PatientList patient;
    if (isEmpty()){
        return;
    }
}

```



```

while(temp!=nullptr&&temp->patient->id!=id){
    temp=temp->next;
}
if(temp==nullptr){
    cout<<"patient not found"<<endl;
    return;
}
temp->patient->disease=newDisease;
temp->patient->doctorName=newDoctor;
patient.updatePatient(id,newDisease,newDoctor);
}
};

```

```

int main(){
    PatientList patientList;
    AppointmentQueue normalQueue;
    EmergencyQueue emergencyQueue;

    int choice;
    bool flag=true;
    while(flag){
        cout<< "\nHospital Patient Management System\n";
        cout<<"1. Add Patient\n2. Delete Patient\n3. Search Patient\n4. Update patients
information.\n";
        cout<<"5. Show patients to be treated.\n6. Show Normal Queue\n";
        cout<<"7. Show Emergency Queue\n8. Serve Next Emergency.\n9. Serve next normal
patient\n10. Show patients that have been discharged after taking treatment\n11. Exit\n";
    }
}

```

```

cout<<"Choose option: ";

cin >> choice;

int id, age, priority,patienttype;

string name, disease, doctor;

Patient* p;

switch (choice) {

    case 1:

        cout<<"Emergency patient or regular patient?(press 1 for emergency or 2 for normal):
",

        cin>>patienttype;

        if(patienttype==1){

            cout<<"Enter Name, Age, Disease, Doctor, Priority: ";

            cin>>name>>age>>disease>>doctor>>priority;

            p=new Patient(name,age,disease,doctor,priority);

            emergencyQueue.addEmergencyPatient(p);

            patientList.addPatient(p);

        }

        else if(patienttype==2){

            cout << "Enter Name, Age, Disease, Doctor: ";

            cin>>name>>age>>disease>>doctor;

            p=new Patient(name,age,disease,doctor);

            normalQueue.addToQueue(p);

            patientList.addPatient(p);

        }

        else{

```

```
    cout<<"Error. Invalid input. Please choose 1 for emergency or 2 for normal."<<endl;

    break;

}

cout<<"Patient added.\n";

break;
```

case 2:

```
    cout<<"Which type of patient do you want to remove from the system, emergency
patient or regular patient?\n(press 1 for emergency or 2 for normal): ";
```

```
    cin>>patienttype;
```

```
    if(patienttype==1){
```

```
        cout<<"Input name for emergency patient: ";
```

```
        cin>>name;
```

```
        cout<<"\nInput ID no for emergency patient: ";
```

```
        cin>>id;
```

```
        emergencyQueue.deleteemrgencypatient(name,id,patientList);
```

```
    }
```

```
    else if(patienttype==2){
```

```
        cout<<"Input ID no for normal patient: ";
```

```
        int id;
```

```
        cin>>id;
```

```
        normalQueue.deletePatient(id,patientList);
```

```
    }
```

```
    else{
```

```
        cout<<"Error. Please press 1 or 2 and try again."<<endl;
```

```

    }

    break;

case 3:

    cout << "Enter ID to search: ";

    cin >> id;

    p = patientList.searchPatient(id);

    if (p && p->priority>0&&p->priority<=2){

        cout<<"Emergency patient found: " << p->name << ", Disease: " << p->disease
        <<p->priority<<" .Lower number indicates that patient is in critical state." <<endl;

    }

    else if(p){

        cout<<"Normal Patient found: "<<p->name<<" , Disease: "<<p->disease<<endl;

    }

    else{

        cout<<"Patient not found.";

    }

    break;

```

```

case 4:

    cout<<"\nIf patient to be updated is in the emergency section press 1 and for normal
    patients press 2: ";

    cin>>patienttype;

    if(patienttype==1){

        cout << "Enter ID to update, New Disease, New Doctor: ";

        cin >> id >> disease >> doctor;

        emergencyQueue.updatePatient(id,disease,doctor);
    }

```

```
        break;
    }
    else if(patienttype==2){
        cout << "Enter ID to update, New Disease, New Doctor: ";
        cin >> id >> disease >> doctor;
        normalQueue.updatePatient(id, disease, doctor);
        break;
    }
    else{
        break;
    }
}
```

case 5:

```
    patientList.displayAllPatients();
    break;
```

case 6:

```
    normalQueue.displayQueue();
    break;
```

case 7:

```
    emergencyQueue.displayEmergencyQueue();
    break;
```

case 8:

```
    emergencyQueue.serveNextEmergency(patientList);
    break;
```

```
case 9:
    normalQueue.serveNextPatient(patientList);
    break;
case 10:
    patientList.showdischarged();
    break;
case 11:
    cout<<"Exiting system."<<endl;
    flag=false;
    break;
default:
    cout<<"Invalid option. Try again\n";
}
}
return 0;
}
```